

```

#include <stdio.h>

int main() {
    int page[10];
    int n, ps, pno, off, frame;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page size: ");
    scanf("%d", &ps);

    printf("Enter frame number for each page:\n");

    for (int i = 0; i < n; i++) {
        printf("Page %d: ", i);
        scanf("%d", &page[i]);
    }

    printf("Enter page number and offset: ");
    scanf("%d%d", &pno, &off);

    frame = page[pno];

    if (frame == -1) {
        printf("Page Fault\n");
    } else {
        int physical_address;

        physical_address = frame * ps + off;

        printf("Frame Number = %d\n", frame);
        printf("Physical Address = %d\n", physical_address);
    }
}

#####
#include <stdio.h>

int main() {
    int numSeg;

    printf("Enter number of segments: ");
    scanf("%d", &numSeg);

    int segTable[10][2]; // [Limit][Base]

    printf("\nEnter Limit and Base Address for each segment:\n");

    for (int i = 0; i < numSeg; i++) {
        printf("Segment %d: ", i);
        scanf("%d %d", &segTable[i][0], &segTable[i][1]);
    }

    int segNo, offset;

    printf("\nEnter Segment Number and Offset: ");
    scanf("%d %d", &segNo, &offset);

    if (segNo < 0 || segNo >= numSeg) {
        printf("Invalid Segment Number\n");
    } else if (offset < 0 || offset >= segTable[segNo][0]) {
        printf("Invalid Offset (exceeds limit)\n");
    } else {
        int base = segTable[segNo][1];
        int physicalAddress = base + offset;

        printf("Base Address = %d\n", base);
        printf("Physical Address = %d\n", physicalAddress);
    }
}

```

```

1 #####
2 #include <stdio.h>
3 int main() {
4     int nof, nor;
5
6     printf("Enter number of frames: ");
7     scanf("%d", &nof);
8
9     printf("Enter number of pages in reference string: ");
10    scanf("%d", &nor);
11
12    int ref[50];
13    printf("Enter reference string:\n");
14
15    for (int i = 0; i < nor; i++) scanf("%d", &ref[i]);
16
17    int frame[10];
18
19    for (int i = 0; i < nof; i++) frame[i] = -1;
20
21    int pageFaults = 0;
22    int victim = 0;
23
24    printf("\nPage\tFrames\n");
25
26    for (int i = 0; i < nor; i++) {
27        int found = 0;
28        // checking for page hit
29        for (int j = 0; j < nof; j++) {
30            if (frame[j] == ref[i]) {
31                found = 1;
32                break;
33            }
34        }
35
36        if (!found) {
37            frame[victim] = ref[i];
38            victim = (victim + 1) % nof;
39            pageFaults++;
40        }
41
42        printf("%d\t", ref[i]);
43        for (int j = 0; j < nof; j++) printf("%d ", frame[j]);
44        printf("\n");
45    }
46    printf("\nTotal Page Faults = %d\n", pageFaults);
47 }
48
49

```

```

1  #include <stdio.h>
2
3  int main() {
4      int nof, nor;
5
6      printf("Enter number of frames: ");
7      scanf("%d", &nof);
8
9      printf("Enter number of pages: ");
10     scanf("%d", &nor);
11
12     int ref[50];
13
14     printf("Enter reference string:\n");
15
16     for (int i = 0; i < nor; i++) scanf("%d", &ref[i]);
17
18     int frame[10];
19
20     for (int i = 0; i < nof; i++) frame[i] = -1;
21
22     int pageFaults = 0;
23
24     printf("\nPage\tFrames\n");
25
26     for (int i = 0; i < nor; i++) {
27         int found = 0;
28         // checking for page hit
29         for (int j = 0; j < nof; j++) {
30             if (frame[j] == ref[i]) {
31                 found = 1;
32                 break;
33             }
34         }
35
36         if (!found) {
37             int pos = -1;
38             // checking for empty frame
39             for (int j = 0; j < nof; j++) {
40                 if (frame[j] == -1) {
41                     pos = j;
42                     break;
43                 }
44             }
45             // Need replacement
46             if (pos == -1) {
47                 int lruIndex = i;
48                 pos = 0;
49
50                 for (int j = 0; j < nof; j++) {
51                     int k;
52
53                     for (k = i - 1; k >= 0; k--) {
54                         if (frame[j] == ref[k]) break;
55                     }
56
57                     if (k < lruIndex) {
58                         lruIndex = k;
59                         pos = j;
60                     }
61                 }
62             }
63
64             frame[pos] = ref[i];
65             pageFaults++;
66         }
67
68         printf("%d\t", ref[i]);
69         for (int j = 0; j < nof; j++) printf("%d ", frame[j]);
70         printf("\n");
71     }
72     printf("\nTotal Page Faults = %d\n", pageFaults);
73 }

```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5
6  int main() {
7      int fd[2];
8      char write_msg[100];
9      char read_msg[100];
10
11     if (pipe(fd) == -1) {
12         perror("Pipe failed");
13         exit(1);
14     }
15
16     pid_t pid;
17     pid = fork();
18
19     if (pid < 0) {
20         perror("Fork failed");
21         exit(1);
22     }
23
24     // Parent Process
25     if (pid > 0) {
26         close(fd[0]); // Close read end
27         printf("Enter message: ");
28         scanf("%s", write_msg);
29         write(fd[1], write_msg, strlen(write_msg) + 1);
30         close(fd[1]);
31     }
32
33     // Child Process
34     else {
35         close(fd[1]); // Close write end
36         read(fd[0], read_msg, sizeof(read_msg));
37         printf("Child received: %s\n", read_msg);
38         close(fd[0]);
39     }
40 }
41 #####
42 #include <stdio.h>
43 #include <stdlib.h>
44 #include <sys/shm.h>
45 #include <string.h>
46
47 int main() {
48     int shmid;
49     shmid = shmget(2345, 1024, 0666 | IPC_CREAT);
50     if (shmid == -1) {
51         perror("shmget");
52         exit(1);
53     }
54
55     char *shared_memory;
56     shared_memory = (char *)shmat(shmid, NULL, 0);
57
58     printf("Shared Memory ID : %d\n", shmid);
59     printf("Process attached at %p\n", shared_memory);
60
61     printf("Enter message : ");
62
63     char message[100];
64     fgets(message, sizeof(message), stdin);
65
66     strcpy(shared_memory, message);
67
68     printf("Written to Shared Memory : %s", shared_memory);
69 }

```

```

1  #include <stdio.h>
2  int main() {
3      int nof, nor;
4      printf("Enter number of frames: ");
5      scanf("%d", &nof);
6      printf("Enter number of pages: ");
7      scanf("%d", &nor);
8      int pages[50];
9
10     printf("Enter reference string:\n");
11
12     for (int i = 0; i < nor; i++) scanf("%d", &pages[i]);
13
14     int frame[10];
15
16     for (int i = 0; i < nof; i++) frame[i] = -1;
17
18     int pageFaults = 0;
19
20     printf("\nPage\tFrames\n");
21
22     for (int i = 0; i < nor; i++) {
23         int found = 0;
24
25         // Check page hit
26         for (int j = 0; j < nof; j++) {
27             if (frame[j] == pages[i]) {
28                 found = 1;
29                 break;
30             }
31         }
32
33         if (!found) {
34             int pos = -1;
35
36             // Find empty frame
37             for (int j = 0; j < nof; j++) {
38                 if (frame[j] == -1) {
39                     pos = j;
40                     break;
41                 }
42             }
43
44             // Need replacement
45             if (pos == -1) {
46                 int farthest = -1;
47
48                 for (int j = 0; j < nof; j++) {
49                     int k;
50
51                     for (k = i + 1; k < nor; k++) {
52                         if (frame[j] == pages[k]) break;
53                     }
54
55                     // Never used again
56                     if (k == nor) {
57                         pos = j;
58                         break;
59                     }
60
61                     if (k > farthest) {
62                         farthest = k;
63                         pos = j;
64                     }
65                 }
66             }
67
68             frame[pos] = pages[i];
69             pageFaults++;
70         }
71
72         printf("%d\t", pages[i]);
73         for (int j = 0; j < nof; j++) printf("%d ", frame[j]);
74         printf("\n");
75     }
76     printf("\nTotal Page Faults = %d\n", pageFaults);
77 }

```

```

1  #include<stdio.h>
2  #include<string.h>
3  #include<fcntl.h>
4  #include<unistd.h>
5  #include<sys/stat.h>
6  #define FIFO_NAME "myfifo"
7  int main(){
8      int fd;
9      char msg[100];
10     mkfifo(FIFO_NAME, 0666);
11     fd = open(FIFO_NAME, O_WRONLY);
12     printf("Enter data: ");
13     fgets(msg, sizeof(msg), stdin);
14     write(fd, msg, strlen(msg) + 1);
15     close(fd);
16 }
17 #####
18 #include<stdio.h>
19 #include<fcntl.h>
20 #include<unistd.h>
21 #include<sys/stat.h>
22 #define FIFO_NAME "myfifo"
23 int main(){
24     int fd;
25     char msg[100];
26     fd = open(FIFO_NAME, O_RDONLY);
27     read(fd, msg, sizeof(msg));
28     printf("Received: %s", msg);
29     close(fd);
30 }
31 #####
32 #include <pthread.h>
33 #include <semaphore.h>
34 #include <stdio.h>
35 #include <unistd.h>
36 int buffer[5];
37 int front = 0, rear = 0;
38 sem_t mutex, full, empty;
39 void *producer(void *arg) {
40     for (int i = 0; i < 10; i++) {
41         sem_wait(&empty);
42         sem_wait(&mutex);
43         rear = (rear + 1) % 5;
44         buffer[rear] = i;
45         printf("Produced: %d\n", i);
46         sleep(1);
47         sem_post(&mutex);
48         sem_post(&full);
49     }
50     return NULL;
51 }
52
53 void *consumer(void *arg) {
54     int item;
55     for (int i = 0; i < 10; i++) {
56         sem_wait(&full);
57         sem_wait(&mutex);
58         front = (front + 1) % 5;
59         item = buffer[front];
60         printf("Consumed: %d\n", item);
61         sleep(1);
62         sem_post(&mutex);
63         sem_post(&empty);
64     }
65     return NULL;
66 }
67 int main() {
68     pthread_t producer_thread, consumer_thread;
69     sem_init(&mutex, 0, 1);
70     sem_init(&full, 0, 0);
71     sem_init(&empty, 0, 5);
72     pthread_create(&producer_thread, NULL, producer, NULL);
73     pthread_create(&consumer_thread, NULL, consumer, NULL);
74     pthread_join(producer_thread, NULL);
75     pthread_join(consumer_thread, NULL);
76 }

```



```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/ipc.h>
5  #include <sys/msg.h>
6
7  #define MAX 512
8
9  struct msg {
10     long type;
11     char text[MAX];
12 };
13
14 int main() {
15     int msgid;
16     msgid = msgget(12345, 0666 | IPC_CREAT);
17     if (msgid == -1) {
18         perror("msgget");
19         exit(1);
20     }
21
22     struct msg m;
23
24     while (1) {
25         printf("Enter message: ");
26         fgets(m.text, MAX, stdin);
27
28         m.type = 1;
29
30         msgsnd(msgid, &m, strlen(m.text) + 1, 0);
31
32         if (strncmp(m.text, "end", 3) == 0) break;
33     }
34 }
35 #####
36 include <stdio.h>
37 #include <stdlib.h>
38 #include <string.h>
39 #include <sys/ipc.h>
40 #include <sys/msg.h>
41
42 #define MAX 512
43
44 struct msg {
45     long type;
46     char text[MAX];
47 };
48
49 int main() {
50     int msgid;
51     msgid = msgget(12345, 0666 | IPC_CREAT);
52
53     if (msgid == -1) {
54         perror("msgget");
55         exit(1);
56     }
57     struct msg m;
58     while (1) {
59         msgrcv(msgid, &m, MAX, 0, 0);
60
61         printf("Received: %s", m.text);
62
63         if (strncmp(m.text, "end", 3) == 0) break;
64     }
65
66     msgctl(msgid, IPC_RMID, NULL);
67 }

```

```

1  #include <stdbool.h>
2  #include <stdio.h>
3
4  #define MAX 100
5
6  int max[MAX][MAX];
7  int allocation[MAX][MAX];
8  int need[MAX][MAX];
9  int available[MAX];
10
11 int numProcesses, numResources;
12
13 void input();
14 void calculateNeed();
15 void display();
16 void bankersAlgorithm();
17
18 int main() {
19     printf("***** Banker's Algorithm *****\n");
20
21     input();
22     calculateNeed();
23     display();
24     bankersAlgorithm();
25     return 0;
26 }
27 void bankersAlgorithm() {
28     int finished[MAX] = {0};
29     int safeSequence[MAX];
30     int count = 0;
31     // run till we cross total processes
32     while (count < numProcesses) {
33         int found = 0;
34         // run for every process
35         for (int i = 0; i < numProcesses; i++) {
36             if (finished[i] == 0) {
37                 int canExecute = 1;
38                 // do we have enough resources to execute this process
39                 for (int j = 0; j < numResources; j++) {
40                     if (need[i][j] > available[j]) {
41                         canExecute = 0;
42                         break;
43                     }
44                 }
45                 // yes we can
46                 if (canExecute == 1) {
47                     for (int j = 0; j < numResources; j++) {
48                         available[j] += allocation[i][j];
49                     }
50
51                     safeSequence[count++] = i;
52                     finished[i] = 1;
53                     found = 1;
54                 }
55             }
56         }
57         // not a single process can execute
58         if (found == 0) {
59             printf("\nSystem is NOT in a safe state (Deadlock possible)\n");
60             return;
61         }
62     }
63     // while loop ends
64     printf("\nSystem is in SAFE state\n");
65     printf("Safe sequence: ");
66
67     for (int i = 0; i < numProcesses; i++) {
68         printf("P%d ", safeSequence[i]);
69     }
70 }
71 void input() {
72     printf("Enter number of processes: ");
73     scanf("%d", &numProcesses);
74
75     printf("Enter number of resources: ");
76     scanf("%d", &numResources);
77

```



```

78     printf("Enter Max matrix:\n");
79     for (int i = 0; i < numProcesses; i++) {
80         for (int j = 0; j < numResources; j++) {
81             scanf("%d", &max[i][j]);
82         }
83     }
84
85     printf("Enter Allocation matrix:\n");
86     for (int i = 0; i < numProcesses; i++) {
87         for (int j = 0; j < numResources; j++) {
88             scanf("%d", &allocation[i][j]);
89         }
90     }
91
92     printf("Enter Available resources:\n");
93     for (int j = 0; j < numResources; j++) {
94         scanf("%d", &available[j]);
95     }
96 }
97
98 void calculateNeed() {
99     for (int i = 0; i < numProcesses; i++) {
100         for (int j = 0; j < numResources; j++) {
101             need[i][j] = max[i][j] - allocation[i][j];
102         }
103     }
104 }
105
106 void display() {
107     printf("\nProcess\tAllocation\tMax\tNeed\n");
108
109     for (int i = 0; i < numProcesses; i++) {
110         printf("P%d\t", i);
111
112         for (int j = 0; j < numResources; j++) printf("%d ", allocation[i][j]);
113
114         printf("\t\t");
115
116         for (int j = 0; j < numResources; j++) printf("%d ", max[i][j]);
117
118         printf("\t");
119
120         for (int j = 0; j < numResources; j++) printf("%d ", need[i][j]);
121
122         printf("\n");
123     }
124
125     printf("Available: ");
126     for (int j = 0; j < numResources; j++) printf("%d ", available[j]);
127
128     printf("\n");
129 }

```